Instituto Superior de Engenharia de Coimbra
Arquiteturas Móveis

Trabalho Prático
Licenciatura Engenharia Informática 2025 / 2026

Duarte Xavier de Oliveira Santos
a2022149622@isec.pt
Gustavo Trigo Costa
a2023145800@isec.pt

1. Introdução.....3
2. Arquitetura e Tecnologias utilizadas.....4
2.1 Tecnologias Utilizadas.....4
2.2 Arquitetura.....4
2.3 Gestão de Assincronismo.....5
3. Implementação e Funcionalidades Principais.....6
3.1. Gestão de Sensores e Deteção de Eventos.....6
3.2. Localização e Geofencing.....8
3.3. Prova Forense e Gravação de Vídeo.....9
3.4. Gestão de Privacidade e Janelas Temporais.....10
3.5. Internacionalização.....11
4. Modelo de dados.....11
5. Oportunidades de Melhoria.....13
5.1. Refinamento do Algoritmo de Queda.....13
5.2. MFA Real (Autenticação Multifator):.....13
5.2. Gestão de Bateria:.....13
6. Conclusão.....14

1. Introdução

Este relatório descreve o desenvolvimento de uma aplicação Android móvel em Kotlin.
O objetivo era desenvolver uma aplicação Android nativa (SafetYSec) que permitiria a monitorização remota de utilizadores (os “Protegidos”) por cuidadores (os “Monitores”). Isto inclui a deteção automática de acidentes e gestão de regras de seguranças (indicadas pelo monitor).

O projeto cumpre os requisitos funcionais de autenticação, gestão de perfis, monitorização via GPS e sensores, recolha de provas forense (vídeo) e internacionalização.

2. Arquitetura e Tecnologias utilizadas

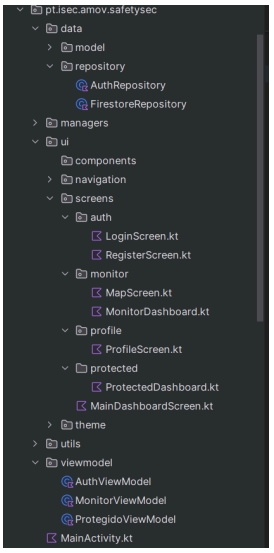
2.1 Tecnologias Utilizadas

- **Linguagem:** Kotlin
- **UI Toolkit:** Jetpack Compose
- **BackEnd:** Firebase (Authentication, Firestore Database e Cloud Storage)
- **Hardware:** Gps (Google Play Services), Acelerómetro e câmara.
- **Programação Assíncrona e Reativa:** Kotlin Coroutines e Kotlin Flows (para gestão eficiente de tarefas em segundo plano e processamento de dados em tempo real).

2.2 Arquitetura

Neste trabalho foi utilizado um padrão MVVM (Model-View-ViewModel) de modo a separar completamente a lógica da interface.

- ***Model/Data:*** Camada responsável pela abstração e comunicação com a base de dados (Firestore), garantindo que os dados persistidos estão sincronizados com a lógica da aplicação..
- ***ViewModel:*** Contém toda a lógica de negócio e gere o estado da UI. É responsável por expor os dados à View e processar as interações do utilizador
- ***View (UI):*** Package exclusivamente para UI (ecrãs em Compose) que apenas observam o estadoornecido pelo ViewModel e reagem às suas alterações, sem conter lógica de negócio complexa.



2.3 Gestão de Assincronismo

Para garantir a fluidez da interface gráfica (UI) e evitar bloqueios na 'Main Thread', foi adotada a biblioteca **Kotlin Coroutines** para todas as operações de entrada/saída (I/O), como chamadas ao Firebase Firestore e operações de disco. Complementarmente, utilizou-se **Kotlin Flows** para a gestão reativa de eventos contínuos. Esta abordagem permite que a aplicação reaja instantaneamente a atualizações de sensores e localização GPS, processando os dados de forma sequencial e segura dentro do ciclo de vida dos ViewModels.

3. Implementação e Funcionalidades Principais

Este capítulo detalha a implementação técnica das funcionalidades nucleares da aplicação SafetYSec, descrevendo os algoritmos e componentes lógicos desenvolvidos para responder aos requisitos de segurança e monitorização.

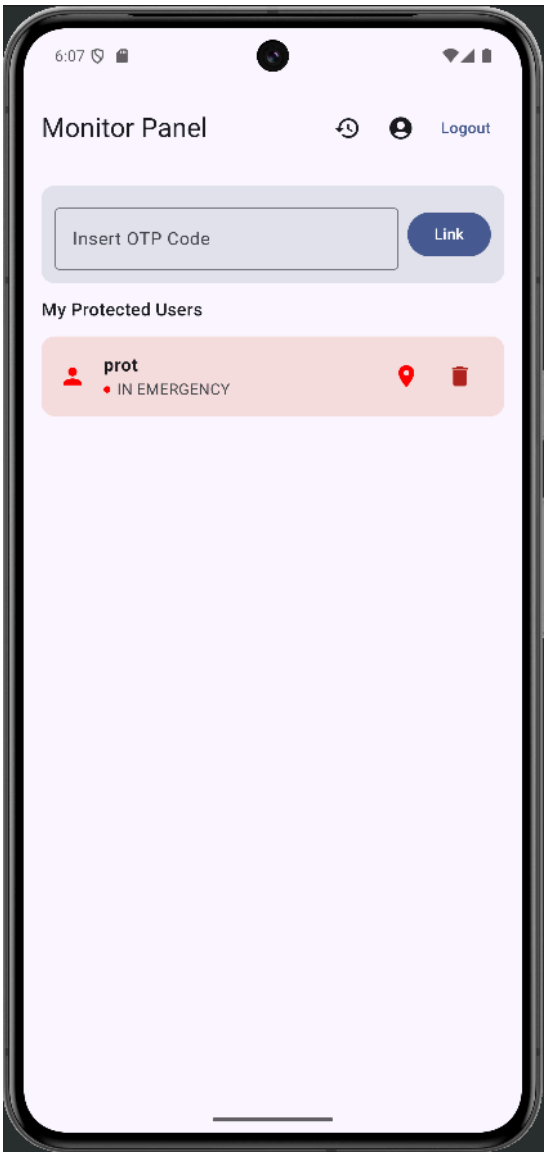
3.1. Gestão de Sensores e Deteção de Eventos

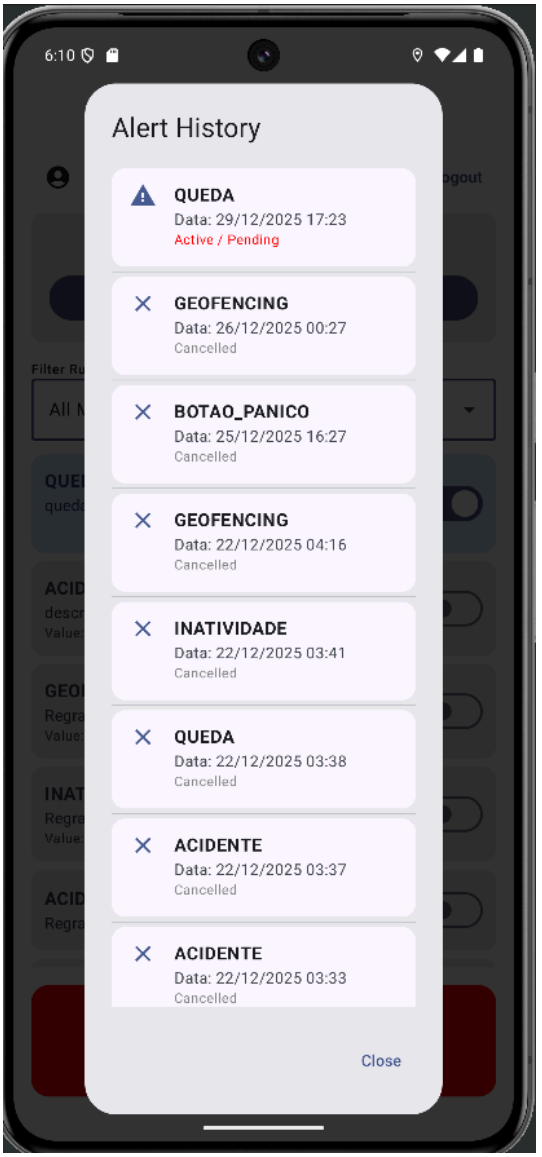
A deteção automática de incidentes é gerida pela classe **SensorManager**, que interage diretamente com o hardware do dispositivo através da API de Sensores do Android. A aplicação subscreve, através de Kotlin Flows, os dados provenientes do Acelerómetro (**Sensor.TYPE_ACCELEROMETER**) para monitorizar as forças G exercidas sobre o telemóvel em tempo real.

O algoritmo implementado distingue dois tipos de eventos críticos:
x

- Queda: Para evitar falsos positivos, o sistema procura um padrão sequencial específico: uma descida abrupta da aceleração para valores próximos de 0G (indicando queda livre ou ausência de peso), seguida imediatamente por um pico de aceleração superior a um limiar configurado (impacto no solo).

- Acidente: Detetado através de uma força G súbita e violenta (acima de 15G nos testes) sem o pré-requisito de queda livre, simulando o impacto de uma colisão rodoviária.

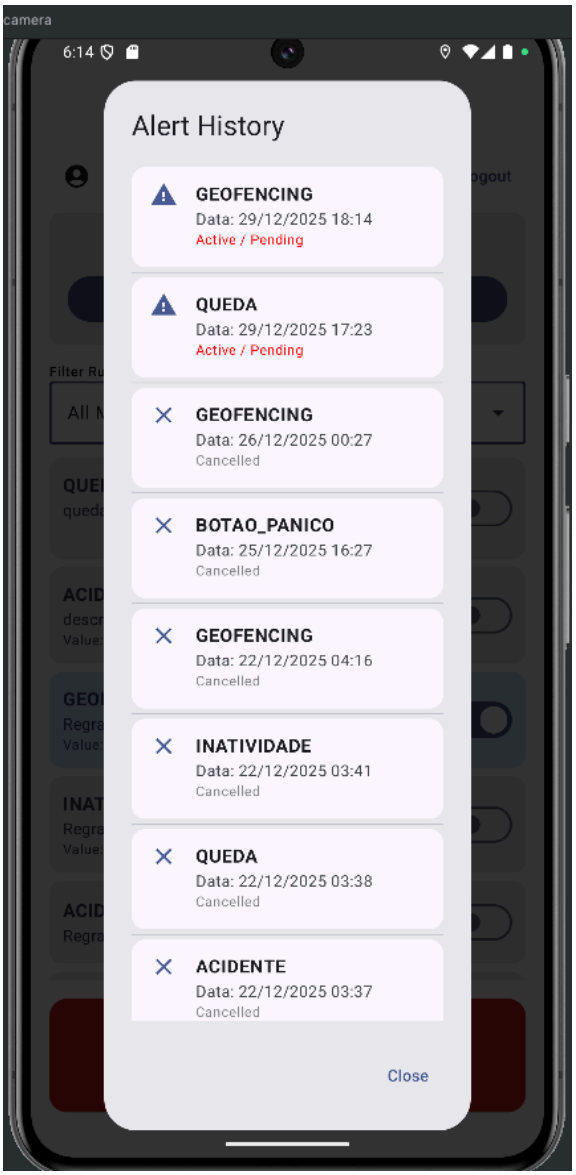

6:07
Monitor Panel
Logout
Insert OTP Code
Link
My Protected Users
prot
IN EMERGENCY


6:10
Alert History
QUEDA
Data: 29/12/2025 17:23
Active / Pending
GEOFENCING
Data: 26/12/2025 00:27
Cancelled
BOTAO_PANICO
Data: 25/12/2025 16:27
Cancelled
GEOFENCING
Data: 22/12/2025 04:16
Cancelled
INATIVIDADE
Data: 22/12/2025 03:41
Cancelled
QUEDA
Data: 22/12/2025 03:38
Cancelled
ACIDENTE
Data: 22/12/2025 03:37
Cancelled
ACIDENTE
Data: 22/12/2025 03:33
Cancelled
Close

3.2. Localização e Geofencing

A monitorização da localização foi encapsulada na classe **LocationManager**, recorrendo ao **FusedLocationProviderClient** da Google para obter coordenadas com elevada precisão e eficiência energética.

A lógica de **Geofencing** (Cercas Virtuais) é processada no **ProtegidoViewModel**. Sempre que a localização é atualizada, o sistema itera sobre as regras ativas do tipo GEOFENCING. É calculada a distância geodésica entre a posição atual do utilizador e o centro da regra definida pelo monitor (latitude/longitude), utilizando o método **Location.distanceBetween**. Caso a distância calculada exceda o raio estipulado na regra, um alerta é despoletado automaticamente.



3.3. Prova Forense e Gravação de Vídeo

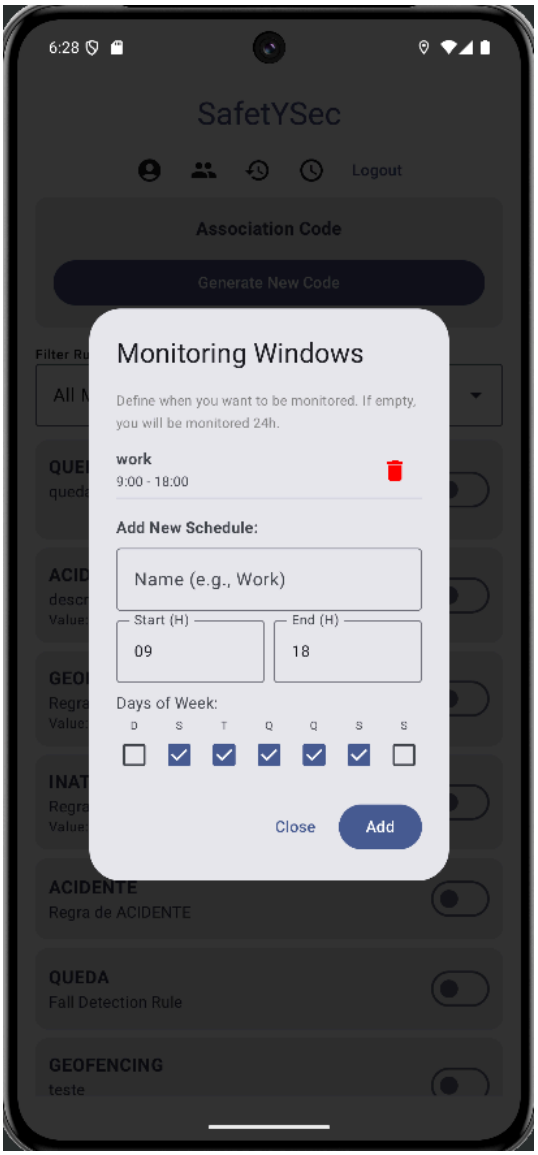
Para garantir o registo de evidências em situações de emergência, foi desenvolvida a classe **CameraManager** utilizando a biblioteca CameraX.

A gravação de vídeo é iniciada automaticamente assim que um alerta é ativado (seja por deteção de sensores ou pelo Botão de Pânico). O sistema gere o ciclo de vida da câmara para operar em segundo plano, não bloqueando a interface do utilizador. Após a captura de 30 segundos de vídeo, o ficheiro é comprimido e enviado de forma assíncrona para o **Firebase Storage**. O URL de acesso ao vídeo é, posteriormente, anexado ao documento do alerta na base de dados (Firestore), permitindo ao Monitor visualizar o contexto do incidente.

3.4. Gestão de Privacidade e Janelas Temporais

Reconhecendo a importância da privacidade do utilizador "Protegido", foi implementado um sistema de controlo temporal através da entidade **TimeWindow**.

Antes de ativar os serviços de monitorização (GPS e Sensores), o **ProtegidoViewModel** verifica se a hora atual e o dia da semana correspondem a uma janela de autorização definida pelo utilizador. Se o utilizador estiver fora do horário estipulado, os serviços de recolha de dados permanecem inativos, garantindo que a monitorização ocorre apenas nos períodos estritamente necessários.



3.5. Internacionalização

A aplicação foi preparada para um ambiente global, suportando nativamente os idiomas Inglês (por omissão) e Português. Toda a interface de utilizador (UI) utiliza referências a recursos de texto (**strings.xml**) em vez de texto fixo no código. O sistema Android encarrega-se de carregar o ficheiro de recursos adequado (**values** ou **values-pt**) com base nas definições de idioma do dispositivo móvel do utilizador, sem necessidade de reinicialização da aplicação.

4. Modelo de dados

A persistência de dados no Cloud Firestore organiza-se em três coleções principais, sendo estas coleções geridas pelo ***FirestoreRepository:***

- **Coleção *Users:*** Armazena os perfis de utilizador. Para permitir a associação Monitor-Protegido, cada documento de utilizador contém listas de IDs (***associatedMonitorsIds e associatedProtegidosIds***). Para isto utilizamos um código OTP (***connectionCode***).

```
35 Usages
class User (
    val id: String = "",
    val email: String = "",
    val name: String = "",

    @get:PropertyName( value = "isMonitor")
    @set:PropertyName( value = "isMonitor")
    var isMonitor: Boolean = false,

    @get:PropertyName( value = "isProtected")
    @set:PropertyName( value = "isProtected")
    var isProtected: Boolean = false,

    val cancellationCode: String = "1234",
    val connectionCode: String? = null,
    val associatedMonitorIds: List <String> = emptyList(),
    val associatedProtegidoIds: List <String> = emptyList(),

    val lastLatitude: Double? = null,
    val lastLongitude: Double? = null,

    //para sabermos se o Protegido está "Offline" há muito tempo
    val lastLocationTime: Long? = null
)
```

- **Coleção *rules:*** Guarda as definições de segurança. Cada regra possui um 'type', um estado ***isActive*** e valores variáveis que dependem do tipo de regra. A relação entre as regras é feita guardando o ***monitorId e o protectedId.***

```
14 Usages
data class Rule (
    val id: String = "",
    val type: RuleType = RuleType.UNKNOWN,
    val description: String = "",

    @get:PropertyName( value = "isActive")
    @set:PropertyName( value = "isActive")
    var isActive: Boolean = false,
    val valueDouble: Double ? = null, // usamos este campo para armazenar a velocidade ou a inatividade
    val latitude: Double? = null,
    val longitude: Double? =null,
    val radius: Double? = null,

    val monitorId: String = "", // para sabermos que monitor criou a regra
    val protectedId: String = "" // para sabermos a que protegido se aplica

)
```

- **Coleção *alerts:*** Regista o histórico de ocorrências. Cada documento é imutável após a criação (excepto para a resolução desse alerta). Cada alert contém o 'type', coordenadas GPS no momento do alerta e o link para vídeo de prova (***videoUrl***).

```
15 Usages
data class Alert (
    val id: String = "",
    val type: RuleType = RuleType.UNKNOWN,
    val userEmail : String = "", //apenas para display
    val protectedId: String = "", //sabermos quem causou o alerta

    //dados do evento
    val date: Date = Date(),
    val latitude: Double = 0.0,
    val longitude: Double = 0.0,

    val solved: Boolean = false, //se o monitor já resolveu o alerta
    val cancelled: Boolean = false, //se o protegido decidiu cancelar o alerta nos 10s

    val videoUrl: String = ""

)
```

5. Oportunidades de Melhoria

5.1. Refinamento do Algoritmo de Queda

Atualmente, a deteção de quedas baseia-se em limiares de aceleração fixos. Uma melhoria futura seria a implementação da **Activity Recognition API** da Google ou modelos de Machine Learning para distinguir melhor uma queda real de movimentos bruscos do telemóvel (ex: atirar o telemóvel para o sofá).

5.2. MFA Real (Autenticação Multifator):

A solução implementada simula o envio de email. Numa versão de produção, integrar-se-ia uma API de envio de emails ou SMS para entregar o código ao utilizador.

5.2. Gestão de Bateria:

O uso contínuo de GPS de alta precisão (PRIORITY_HIGH_ACCURACY no LocationManager.kt) consome muita bateria. Uma melhoria seria implementar um algoritmo que ajusta a frequência de atualização com base na velocidade de movimento ou bateria restante.

6. Conclusão

Concluindo o projeto **SafetYSec** atingiu todos os objetivos propostos no enunciado, resultando assim numa solução de segurança pessoal funcional e robusta.
A utilização de componentes de arquitetura Android modernos permitiu criar um código limpo, testável e preparado para uma possível evolução futura.
Funcionalidades críticas como a deteção de acidentes e a recolha de provas forenses foram implementadas com sucesso.